

LAPORAN PRAKTIKUM

STRUKTUR DATA

“ SORTING”



disusun Oleh:

FILZI JELILA INDA ROBBANI

(2511533019)

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan metode pengolahan data sorting (proses pengurutan) lanjutan pada Java .

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan.....	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Langkah Kerja Praktikum	6
2.1.1 Membuat kelas pertama (ShellSort).....	6
2.1.2 Membuat kelas kedua (QuickSort)	7
2.1.3 Membuat program ketiga (MergeSort)	10
2.1.4 Membuat program keempat BubleSortGUI	12
2.1.5 Membuat program keempat BubleSortGUI	19
2.2 Tugas Praktikum	27
2.2.1 Deskripsi Tugas	27
2.2.2 Kelas ADT Lagu.....	27
2.2.3 Kelas Utama	28
BAB III PENUTUP.....	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya telah dipelajari beberapa algoritma sorting, yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Ketiga algoritma tersebut digunakan untuk mengurutkan data dengan metode dan karakteristik yang berbeda sehingga mahasiswa dapat memahami konsep dasar pengurutan data.

Pada praktikum kali ini, pembelajaran dilanjutkan dengan mempelajari algoritma sorting yang lebih kompleks, yaitu Shell Sort, Quick Sort, dan Merge Sort. Algoritma-algoritma tersebut memiliki mekanisme pengurutan yang lebih efisien dibandingkan algoritma sorting sederhana pada beberapa kondisi tertentu. Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara kerja, implementasi, serta perbandingan efisiensi dari masing-masing algoritma dalam mengurutkan data.

Selain itu, praktikum ini bertujuan untuk meningkatkan kemampuan mahasiswa dalam menerapkan konsep struktur data dan algoritma ke dalam bahasa pemrograman Java sehingga dapat menyelesaikan permasalahan pengolahan data secara lebih efektif dan efisien.

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

- a. Memahami konsep dasar algoritma pengurutan (sorting).
- b. Mempelajari cara kerja algoritma Shell Sort, Quick Sort, dan Merge Sort.
- c. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
- d. Menganalisis hasil pengurutan data sebelum dan sesudah proses sorting.
- e. Mengetahui efisiensi dan karakteristik masing-masing algoritma sorting.

1.3 Manfaat

- a. Menambah pemahaman mengenai konsep struktur data dan algoritma sorting.
- b. Melatih kemampuan pemrograman Java dalam mengimplementasikan algoritma pengurutan.
- c. Membantu memahami perbedaan cara kerja serta performa Shell Sort, Quick Sort, dan Merge Sort.
- d. Meningkatkan kemampuan analisis dalam memilih algoritma yang sesuai untuk suatu permasalahan.
- e. Menjadi dasar dalam mempelajari algoritma dan struktur data yang lebih kompleks pada materi berikutnya..
- f. Meningkatkan kemampuan pembuatan aplikasi berbasis GUI menggunakan Java Swing.

BAB II

PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (ShellSort)

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama Pekan8_nim terus pilih “New”, pilih class. Buat nama class “ShellSort_2511533019”.
- 2) Lalu, masukkan syntax seperti berikut.

```
3) package Pekan8_2511533019;
4)
5) public class ShellSort_2511533019 {
6)
7)     public static void ShellSort_3019(int[] A) {
8)         // TODO Auto-generated method stub
9)         int n_3019 = A.length;
10)        int gap_3019 = n_3019 / 2;
11)        while (gap_3019 > 0) {
12)            for (int i_3019 = gap_3019; i_3019 <
n_3019; i_3019++) {
13)                int temp_3019 = A[i_3019];
14)                int j_3019 = i_3019;
15)                while (j_3019 >= gap_3019 &&
A[j_3019 - gap_3019] > temp_3019) {
16)                    A[j_3019] = A[j_3019 -
gap_3019];
17)                    j_3019 = j_3019 -
gap_3019;
18)                }
19)                A[j_3019] = temp_3019;
20)            }
21)            gap_3019 = gap_3019 / 2;
22)        }
23)    }
24)
25)    public static void main (String[] args) {
26)        int[] data_3019 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
27)
28)        System.out.print ("Sebelum: ");
29)        printArray_3019(data_3019);
30)
31)        ShellSort_3019(data_3019);
32)
33)        System.out.print("Sesudah (Shell Sort): ");
34)        printArray_3019(data_3019);
35)    }
36)
37)    public static void printArray_3019(int[] arr) {
```

```

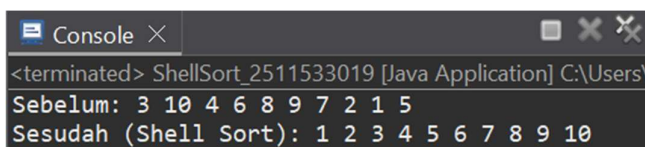
38)   for (int i_3019 : arr) System.out.print(i_3019 + "
    ");
39)   System.out.println();
40)   }
41) }

```

Program pertama yaitu ShellSort_2511533019 digunakan untuk mengimplementasikan algoritma Shell Sort. Pada algoritma ini, proses pengurutan dilakukan dengan membandingkan elemen-elemen yang memiliki jarak tertentu (gap). Nilai gap pada awalnya ditentukan sebesar setengah dari panjang array, kemudian secara bertahap diperkecil hingga bernilai 1. Dengan cara ini, elemen yang berada jauh dari posisi seharusnya dapat dipindahkan lebih cepat dibandingkan metode insertion sort biasa.

Method ShellSort_3019() menerima parameter berupa array bertipe integer yang akan diurutkan. Variabel gap_3019 digunakan untuk menentukan jarak perbandingan antar elemen, sedangkan temp_3019 digunakan sebagai penyimpan sementara saat proses perpindahan data. Variabel j_3019 digunakan untuk menentukan posisi elemen yang sedang dibandingkan. Setelah proses pengurutan selesai, method main() berfungsi menampilkan data sebelum dan sesudah proses sorting dilakukan menggunakan method printArray_3019().

42) Run Program, lalu akan muncul output seperti berikut.



```

<terminated> ShellSort_2511533019 [Java Application] C:\Users\
Sebelum: 3 10 4 6 8 9 7 2 1 5
Setelah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

2.1.2 Membuat kelas kedua (QuickSort)

- 1) Klik kanan pada package Pekan8_nim terus pilih “New”, pilih class. Buat nama class “QuickSort_2511533019”.
- 2) Masukkan syntax seperti berikut.

```

1) package Pekan8_2511533019;
2)
3) public class QuickSort_2511533019 {
4)

```

```

5)    public static void swap_3019(int[] arr_3019, int
      i_3019, int j_3019) {
6)        // TODO Auto-generated method stub
7)        int temp_3019 = arr_3019[i_3019];
8)        arr_3019[i_3019]= arr_3019[j_3019];
9)        arr_3019[j_3019]= temp_3019;
10)   }
11)   // Metode tambahan untuk mengatur pivot menggunakan
      Median-of-Three
12)   static void medianOfThree_3019 (int[] arr_3019, int
      low_3019, int high_3019) {
13)       int mid_3019 = low_3019 + (high_3019 -
      low_3019) / 2;
14)
15)       //Urutkan elemen low, mid, dan high
16)       if (arr_3019 [low_3019] > arr_3019[mid_3019])
      {
17)           swap_3019(arr_3019, low_3019,
      mid_3019);
18)       }
19)       if(arr_3019[low_3019] > arr_3019[high_3019])
      {
20)           swap_3019(arr_3019, low_3019,
      high_3019);
21)       }
22)       if(arr_3019[mid_3019]> arr_3019[high_3019]) {
23)           swap_3019 (arr_3019, mid_3019,
      high_3019);
24)       }
25)       swap_3019(arr_3019, mid_3019, high_3019);
26)   }
27)   static int partition (int[] arr_3019, int low_3019,
      int high_3019) {
28)       //Panggil fungsi medianofThree sebelum menentukan
      pivot
29)       medianOfThree_3019(arr_3019, low_3019, high_3019);
30)
31)       int pivot_3019 = arr_3019 [high_3019]; // Sekarang
      arc[high] sudah berisi nilai median
32)       int i_3019 = (low_3019 - 1);
33)
34)       for(int j_3019= low_3019; j_3019<= high_3019-1;
      j_3019++) {
35)           // Jika elemen saat ini lebih kecil dari atau
      sama
36)           if (arr_3019[j_3019] < pivot_3019) {
37)               // Increment indeks elemen yang Lebih
      kecil
38)               i_3019++;
39)               swap_3019(arr_3019, i_3019,j_3019);
40)           }
41)       }
42)       swap_3019(arr_3019, i_3019 + 1, high_3019);
43)       return (i_3019+ 1);
44)   }
45)   static void quickSort_3019 (int[] arr_3019, int
      low_3019, int high_3019) {

```



```

46)         if (low_3019 < high_3019) {
47)             int pi_3019 = partition(arr_3019,
low_3019, high_3019);
48)             quickSort_3019(arr_3019, low_3019,
pi_3019 - 1);
49)             quickSort_3019 (arr_3019, pi_3019 + 1,
high_3019);
50)         }
51)     }
52)
53)     public static void printArr_3019 (int[] arr_3019) {
54)         for(int i_3019 = 0; i_3019 < arr_3019.length;
i_3019++) {
55)             System.out.print(arr_3019[i_3019] + "
");
56)         }
57)         System.out.println();
58)     }
59)
60)     public static void main (String[] args){
61)         int[] arr_3019 = { 10, 7, 8, 9, 1, 5};
62)         int N_3019 = arr_3019.length;
63)         System.out.print ("Data sebelum diurutkan:
");
64)         printArr_3019(arr_3019);
65)
66)         quickSort_3019 (arr_3019, 0, N_3019 - 1);
67)
68)         System.out.print ("Data Terurut quicksort: ")
;
69)         printArr_3019(arr_3019);
70)     }
71) }

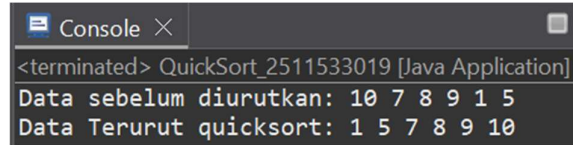
```

Program kedua QuickSort_2511533019 digunakan untuk mengimplementasikan algoritma Quick Sort. Algoritma ini bekerja dengan memilih sebuah elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Proses ini dilakukan secara rekursif hingga seluruh data berada pada urutan yang benar.

Method quickSort_3019() digunakan sebagai method utama yang menjalankan proses pengurutan secara rekursif. Method partition() berfungsi membagi array berdasarkan nilai pivot, sedangkan method medianOfThree_3019() digunakan untuk menentukan pivot menggunakan metode Median of Three agar performa pengurutan lebih baik. Method swap_3019() digunakan untuk menukar posisi dua elemen dalam array. Setelah proses sorting selesai, method

main() menampilkan data sebelum dan sesudah pengurutan melalui method printArr_3019().

- 3) Run program maka akan tampil output seperti berikut.



```
<terminated> QuickSort_2511533019 [Java Application]
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

2.1.3 Membuat program ketiga (MergeSort)

- 1) Klik kanan pada package Pekan8_nim terus pilih “New”, pilih class. Buat nama class “MergeSort_2511533019”.
- 2) Lalu buat/masukkan syntax seperti berikut.

```
1) package Pekan8_2511533019;
2)
3) public class MergeSort_2511533019 {
4)     void merge_3019(int arr_3019[], int l_3019, int
m_3019, int r_3019) {
5)         int n1_3019 = m_3019-l_3019 + 1;
6)         int n2_3019 = r_3019 - m_3019;
7)         /* create temp arrays */
8)         int L_3019[] = new int[n1_3019];
9)         int R_3019[] = new int[n2_3019];
10)        /* copy data to temp arrays */
11)        for (int i_3019 = 0; i_3019 < n1_3019;
++i_3019)
12)            L_3019[i_3019]= arr_3019[l_3019 +
i_3019];
13)        for (int j_3019= 0; j_3019 < n2_3019;
++j_3019)
14)            R_3019[j_3019]= arr_3019[m_3019+ 1 +
j_3019];
15)        int i_3019=0, j_3019= 0;
16)        // initial index of merged subarray array
17)        int k_3019 = l_3019;
18)        while (i_3019<n1_3019 && j_3019<n2_3019) {
19)            if(L_3019[i_3019] <= R_3019[j_3019]) {
20)                arr_3019[k_3019] =
L_3019[i_3019];
21)                i_3019++;
22)            }else {
23)                arr_3019[k_3019] = R_3019[j_3019];
24)                j_3019++;
25)            }
26)            k_3019++;
27)        }
28)        /*Copy remaining elements of L[] if any */
29)        while (i_3019 < n1_3019) {
30)            arr_3019[k_3019] = L_3019[i_3019];
31)            i_3019++;
```

```

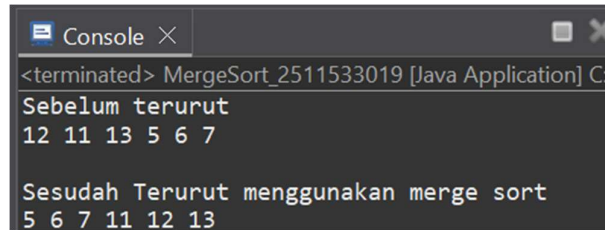
32)         k_3019++;
33)     }
34)     /* copy remaining elements of R[] if any */
35)     while (j_3019 < n2_3019) {
36)         arr_3019[k_3019] = R_3019[j_3019];
37)         j_3019++;
38)         k_3019++;
39)     }
40) }
41) void sort_3019(int arr_3019[], int l_3019, int
r_3019) {
42)     if (l_3019 < r_3019) {
43)         // Find the middle point
44)         int m_3019 = (l_3019 + r_3019) / 2;
45)         // Sort first and second halves
46)         sort_3019(arr_3019, l_3019, m_3019);
47)         sort_3019(arr_3019, m_3019 + 1,
r_3019);
48)         // Merge the sorted halves
49)         merge_3019(arr_3019, l_3019, m_3019,
r_3019);
50)     }
51) }
52) /* A utility function to print array of size n */
53) static void printArray_3019 (int arr_3019[]) {
54)     int n_3019 = arr_3019.length;
55)     for (int i_3019 = 0; i_3019 < n_3019;
++i_3019)
56)         System.out.print(arr_3019[i_3019] + "
");
57)     System.out.println();
58) }
59) public static void main (String args[]) {
60)     int arr_3019[] = { 12, 11, 13, 5, 6, 7 };
61)     System.out.println ("Sebelum terurut");
62)     printArray_3019(arr_3019);
63)     MergeSort_2511533019 ob_3019 = new
MergeSort_2511533019();
64)     ob_3019.sort_3019 (arr_3019, 0,
arr_3019.length - 1);
65)     System.out.println("\nSesudah Terurut
menggunakan merge sort");
66)     printArray_3019(arr_3019);
67) }
68) }

```

Program ketiga yaitu MergeSort_2511533019 digunakan untuk mengimplementasikan algoritma Merge Sort. Algoritma ini menggunakan metode divide and conquer, yaitu membagi array menjadi beberapa bagian yang lebih kecil hingga tersisa satu elemen, kemudian menggabungkannya kembali dalam keadaan terurut.

Method `sort_3019()` digunakan untuk membagi array secara rekursif menjadi dua bagian. Setelah pembagian selesai, method `merge_3019()` bertugas menggabungkan kembali dua bagian array yang telah terurut menjadi satu array yang lebih besar dan tetap terurut. Pada proses penggabungan digunakan array sementara `L_3019` dan `R_3019` untuk menyimpan data dari masing-masing bagian. Method `main()` digunakan untuk menampilkan data sebelum dan sesudah proses pengurutan menggunakan method `printArray_3019()`.

- 3) Run program hingga keluar output seperti berikut, apabila tidak seperti pada gambar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.



```
<terminated> MergeSort_2511533019 [Java Application] C:\n
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge sort
5 6 7 11 12 13
```

2.1.4 Membuat program keempat BubleSortGUI

- 1) Klik kanan package `Pekan8_nim` dan pilih “New”, pilih others, scroll kebawah dan klik `JFram` pada Swing Designer. Lalu next bikin nama “`BubleSortGUI_2511533019`”.
- 2) Tambahkan syntax seperti berikut untuk membangun GUI sesuai dengan tampilan yang diinginkan serta logika sorting yang akan digunakan.

```
1) package Pekan8_2511533019;
2)
3) import java.awt.BorderLayout;
4) import java.awt.Color;
5) import java.awt.Dimension;
6) import java.awt.EventQueue;
7) import java.awt.FlowLayout;
8) import java.awt.Font;
9) import java.lang.reflect.Array;
10)
11) import javax.swing.BorderFactory;
12) import javax.swing.JButton;
```

```

13) import javax.swing.JFrame;
14) import javax.swing.JLabel;
15) import javax.swing.JOptionPane;
16) import javax.swing.JPanel;
17) import javax.swing.JScrollPane;
18) import javax.swing.JTextArea;
19) import javax.swing.JTextField;
20) import javax.swing.SwingConstants;
21) import javax.swing.SwingUtilities;
22) import javax.swing.border.EmptyBorder;
23)
24) public class BubbleSortGUI_2511533019 extends JFrame
    {
25)
26)     private static final long serialVersionUID = 1L;
27)     private int[] array_3019;
28)     private JLabel[] labelArray_3019;
29)     private JButton stepButton_3019, resetButton_3019,
        setButton_3019;
30)     private JTextField inputField_3019;
31)     private JPanel panelArray_3019;
32)     private JTextArea stepArea_3019;
33)
34)     private int i_3019 = 1, j_3019;
35)     private boolean sorting_3019 = false;
36)     private int stepCount_3019 = 1;
37)
38)     /**
39)      * Create the frame.
40)      */
41)     public BubbleSortGUI_2511533019() {
42)         setTitle("Bubble Sort Langkah per Langkah");
43)         setSize(750, 400);
44)
45)         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46)         setLocationRelativeTo(null);
47)         setLayout(new BorderLayout());
48)
49)         // Panel input
50)         JPanel inputPanel_3019 = new JPanel(new
            FlowLayout());
51)         inputField_3019 = new JTextField(30);
52)         setButton_3019 = new JButton("Set Array");
53)         inputPanel_3019.add(new JLabel("Masukkan
            angka (pisahkan dengan koma):"));
54)         inputPanel_3019.add(inputField_3019);
55)         inputPanel_3019.add(setButton_3019);
56)
57)         // Panel array visual
58)         panelArray_3019 = new JPanel();
59)         panelArray_3019.setLayout(new FlowLayout());
60)
61)         // Panel kontrol
62)         JPanel controlPanel_3019 = new JPanel();
63)         stepButton_3019 = new JButton("Langkah
            Selanjutnya");
64)         resetButton_3019 = new JButton("Reset");

```

```

64)         stepButton_3019 .setEnabled(false);
65)         controlPanel_3019 .add(stepButton_3019 );
66)         controlPanel_3019 .add(resetButton_3019 );
67)
68)         // Area teks untuk log langkah-langkah
69)         stepArea_3019 = new JTextArea(8, 60);
70)         stepArea_3019 .setEditable(false);
71)         stepArea_3019 .setFont(new Font("Monospaced",
Font.PLAIN, 14));
72)         JScrollPane scrollPane_3019 = new
JScrollPane(stepArea_3019 );
73)
74)         // Tambahkan panel ke frame
75)         add(inputPanel_3019 , BorderLayout.NORTH);
76)         add(panelArray_3019 , BorderLayout.CENTER);
77)         add(controlPanel_3019 , BorderLayout.SOUTH);
78)         add(scrollPane_3019 , BorderLayout.EAST);
79)
80)         // Event Set Array
81)         setButton_3019.addActionListener(e ->
setArrayFromInput_3019());
82)
83)         // Event Langkah Selanjutnya
84)         stepButton_3019.addActionListener(e ->
performStep_3019());
85)
86)         // Event Reset
87)         resetButton_3019.addActionListener(e ->
reset_3019());
88)
89)     }
90)
91)     private void setArrayFromInput_3019() {
92)         String text_3019 =
inputField_3019.getText().trim();
93)         if (text_3019.isEmpty()) return;
94)         String[] parts_3019 = text_3019.split(",");
95)         array_3019 = new int[parts_3019.length];
96)         try {
97)             for(int k_3019 = 0; k_3019 <
parts_3019.length; k_3019++) {
98)                 array_3019[k_3019] =
Integer.parseInt(parts_3019[k_3019].trim());
99)             }
100)        } catch (NumberFormatException
e) {
101)            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka " + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
102)            return;
103)        }
104)        i_3019 = 0;
105)        j_3019 = 0;
106)        stepCount_3019 = 1;
107)        sorting_3019 = true;
108)        stepButton_3019.setEnabled(true);

```

```

109)         stepArea_3019.setText("");
110)         panelArray_3019.removeAll();
111)         labelArray_3019 = new
            JLabel[array_3019.length];
112)         for (int k_3019 = 0; k_3019 <
            array_3019.length; k_3019++) {
113)             labelArray_3019[k_3019] = new
                JLabel(String.valueOf(array_3019[k_3019]));
114)             labelArray_3019[k_3019].setFont
                (new Font ("Arial", Font.BOLD, 24));
115)             labelArray_3019[k_3019].setOpaque(true);
116)             labelArray_3019[k_3019].setBackground(Color.WHITE)
            ;
117)             labelArray_3019[k_3019].setBorder(BorderFactory.cr
                eateLineBorder(Color.BLACK));
118)             labelArray_3019[k_3019].setPreferredSize (new
                Dimension(50, 50));
119)             labelArray_3019[k_3019].setHorizontalAlignment(Swi
                ngConstants.CENTER);
120)             panelArray_3019.add
                (labelArray_3019[k_3019]);
121)             }
122)             panelArray_3019.revalidate();
123)             panelArray_3019.repaint();
124)             }
125)             private void performStep_3019() {
126)                 if (!sorting_3019 || i_3019 >=
                    array_3019.length - 1) {
127)                     sorting_3019 = false;
128)                     stepButton_3019.setEnabled(false);
129)                     JOptionPane.showMessageDialog(this, "Sorting
                        selesai!");
130)                     return;
131)                 }
132)                 resetHighlights_3019();
133)                 StringBuilder stepLog = new
                    StringBuilder();
134)                 labelArray_3019[j_3019].setBackground(Color.CYAN);
135)                 labelArray_3019[j_3019 +
                    1].setBackground(Color.CYAN);
136)                 if (array_3019[j_3019] >
                    array_3019[j_3019 + 1]) {
137)                     // Swap
138)                     int temp_3019 =
                        array_3019[j_3019];
139)                     array_3019[j_3019] =
                        array_3019[j_3019 + 1];
140)                     array_3019[j_3019 + 1] =
                        temp_3019;

```

```

141)         labelArray_3019[j_3019].setBackground(Color.RED);
142)         labelArray_3019[j_3019 +
143)         1].setBackground(Color.RED);
144)         stepLog.append("Langkah
145)         ").append(stepCount_3019).append(": Menukar elemen
146)         ke-").append(j_3019).append("
147)         ").append(array_3019[j_3019 + 1]).append(") dengan
148)         ke-").append(j_3019 + 1).append("
149)         ").append(array_3019[j_3019]).append(")\n");
150)         } else {
151)         stepLog.append("Langkah
152)         ").append(stepCount_3019).append(": Tidak ada
153)         pertukaran antara ke-").append(j_3019).append(" dan
154)         ke-").append(j_3019 + 1).append(")\n");
155)         }
156)         stepLog.append("Hasil:
157)         ").append(arrayToString_3019(array_3019)).append("\n
158)         \n");
159)         stepArea_3019.append(stepLog.toString());
160)         updateLabels_3019();
161)         j_3019++;
162)         if (j_3019 >= array_3019.length -
163)         i_3019 - 1) {
164)         j_3019 = 0;
165)         i_3019++;
166)         }
167)         stepCount_3019++;
168)         if (i_3019 >= array_3019.length -
169)         1) {
170)         sorting_3019 = false;
171)         stepButton_3019.setEnabled(false);
172)         JOptionPane.showMessageDialog(this, "Sorting
173)         selesai!");
174)         }
175)         }
176)         private void updateLabels_3019() {
177)         for (int k_3019 = 0; k_3019 <
178)         array_3019.length; k_3019++) {
179)         labelArray_3019[k_3019].setText(String.valueOf(arr
180)         ay_3019[k_3019]));
181)         }
182)         }
183)         private void resetHighlights_3019() {
184)         for (JLabel label_3019 :
185)         labelArray_3019) {
186)         label_3019.setBackground(Color.WHITE);
187)         }
188)         }
189)         private void reset_3019() {
190)         inputField_3019.setText("");
191)         panelArray_3019.removeAll();

```



```

175)                panelArray_3019.revalidate();
176)                panelArray_3019.repaint();
177)                stepArea_3019.setText("");
178)
179)                stepButton_3019.setEnabled(false);
180)                sorting_3019 = false;
181)                i_3019 = 0;
182)                j_3019 = 0;
183)                stepCount_3019 = 1;
184)                }
185)                private String arrayToString_3019(int[]
arr) {
186)                StringBuilder sb_3019 = new
StringBuilder();
187)                for (int k_3019 = 0; k_3019 <
arr.length; k_3019++) {
188)                sb_3019.append(arr[k_3019]);
189)                if (k_3019 < arr.length -
1) sb_3019.append(", ");
190)                }
191)                return sb_3019.toString();
192)                }
193)                public static void main(String[] args)
{
194)                {
195)                SwingUtilities.invokeLater(() ->
{
196)                BubbleSortGUI_2511533019
gui = new BubbleSortGUI_2511533019();
197)                gui.setVisible(true);
198)                });
199)                }
200)        }

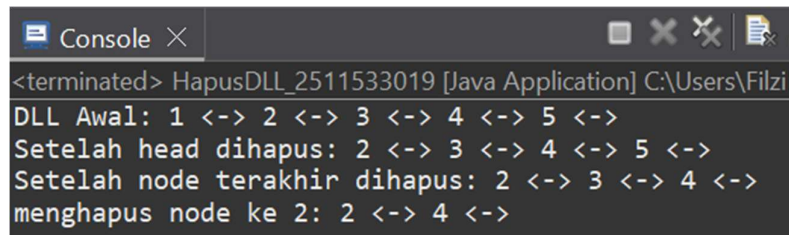
```

Program keempat yaitu BubbleSortGUI_2511533019 merupakan implementasi algoritma Bubble Sort dalam bentuk antarmuka grafis (GUI). Struktur dasar program ini mengadopsi program InsertSortGUI yang telah dibuat pada praktikum sebelumnya, seperti penggunaan komponen GUI berupa JFrame, JPanel, JButton, JTextField, JLabel, dan JTextArea. Perbedaan utama terdapat pada logika proses pengurutan yang diterapkan.

Method setArrayFromInput_3019() digunakan untuk menerima input angka dari user, mengubahnya menjadi array integer, dan menampilkan data tersebut dalam bentuk visual menggunakan label. Setelah data berhasil dimasukkan, proses sorting dapat dilakukan secara bertahap melalui method performStep_3019(). Pada method ini, algoritma Bubble Sort bekerja dengan

membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya tidak sesuai. Setiap langkah pengurutan ditampilkan pada area log serta divisualisasikan menggunakan perubahan warna pada elemen yang sedang dibandingkan. Selain itu, method `reset_3019()` digunakan untuk mengembalikan tampilan program ke kondisi awal sehingga user dapat melakukan simulasi kembali.

- 3) Run program hingga mengeluarkan tampilan seperti berikut.

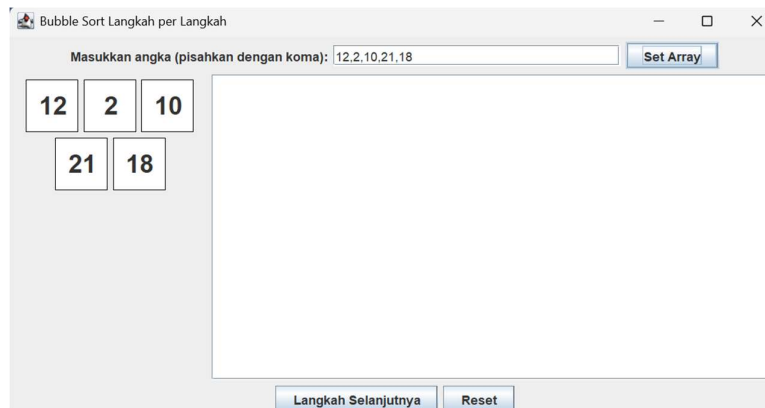


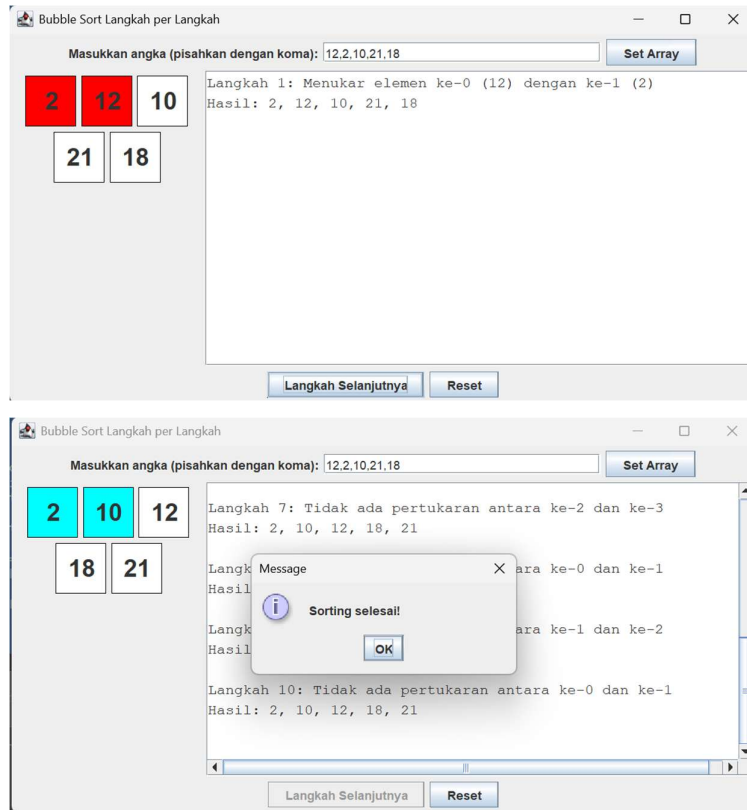
```
<terminated> HapusDLL_2511533019 [Java Application] C:\Users\Filzi
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->
```

Setelah di run, pada awal keluar tampilan untuk bisa melakukan pengurutan angka secara instan dengan metode Buble Sort.

Nah sebagai user inputkan beberapa nilai/angka acak yang ingin kita cari urutan nilai/angkanya. Lalu klik tombol untuk set array dan klik tombol langkah selanjutnya hingga keluar tampilan bahwa sorting selesai.

Sebagai contoh saya memasukkan nilai sebagai berikut:





2.1.5 Membuat program keempat BubleSortGUI

- 1) Klik kanan package Pekan8_nim dan pilih “New”, pilih others, scroll kebawah dan klik JFrame pada Swing Designer. Lalu next bikin nama “MergeSortGUI_2511533019”.
- 2) Tambahkan syntax seperti berikut untuk membangun GUI sesuai dengan tampilan yang diinginkan serta logika sorting yang akan digunakan.

```

1) package Pekan8_2511533019;
2)
3) import java.awt.BorderLayout;
4) import java.awt.Color;
5) import java.awt.Dimension;
6) import java.awt.FlowLayout;
7) import java.awt.Font;
8)
9) import javax.swing.BorderFactory;
10) import javax.swing.JButton;
11) import javax.swing.JFrame;
12) import javax.swing.JLabel;
13) import javax.swing.JOptionPane;

```

```

14) import javax.swing.JPanel;
15) import javax.swing.JScrollPane;
16) import javax.swing.JTextArea;
17) import javax.swing.JTextField;
18) import javax.swing.SwingConstants;
19) import javax.swing.SwingUtilities;
20)
21) import java.util.LinkedList;
22) import java.util.Queue;
23)
24) public class MergeSortGUI_2511533019 extends JFrame
    {
25)
26)     private static final long serialVersionUID =
        1L;
27)     private int[] array_3019;
28)     private JLabel[] labelArray_3019;
29)     private JButton stepButton_3019,
        resetButton_3019, setButton_3019;
30)     private JTextField inputField_3019;
31)     private JPanel panelArray_3019;
32)     private JTextArea stepArea_3019;
33)
34)     private Queue<int[]> mergeQueue_3019 = new
        LinkedList<>();
35)     private int stepCount_3019 = 1;
36)     private boolean isMerging_3019 = false;
37)     private boolean copying_3019 = false;
38)
39)     private int[] temp_3019;
40)     private int left_3019, mid_3019, right_3019;
41)     private int i_3019, j_3019, k_3019;
42)
43)     public MergeSortGUI_2511533019() {
44)         setTitle("Merge Sort Langkah per
            Langkah");
45)         setSize(750, 400);
46)         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
            );
47)         setLocationRelativeTo(null);
48)         setLayout(new BorderLayout());
49)
50)         // panel input
51)         JPanel inputPanel_3019 = new JPanel(new
            FlowLayout());
52)         inputField_3019 = new JTextField(30);
53)         setButton_3019 = new JButton("Set
            Array");
54)         inputPanel_3019.add(new JLabel("Masukan
            angka (pisahkan dengan koma)"));
55)         inputPanel_3019.add(inputField_3019);
56)         inputPanel_3019.add(setButton_3019);
57)
58)         // panel array visual
59)         panelArray_3019 = new JPanel();

```

```

60)         panelArray_3019.setLayout(new
        FlowLayout());
61)
62)         // panel kontrol
63)         JPanel controlPanel_3019 = new
        JPanel();
64)         stepButton_3019 = new JButton("Langkah
        selanjutnya");
65)         resetButton_3019 = new
        JButton("Reset");
66)         controlPanel_3019.add(stepButton_3019);
67)
        controlPanel_3019.add(resetButton_3019);
68)
69)         // area text untuk log langkah langkah
70)         stepArea_3019 = new JTextArea(8, 60);
71)         stepArea_3019.setEditable(false);
72)         stepArea_3019.setFont(new
        Font("Monospaced", Font.PLAIN, 14));
73)         JScrollPane scrollPane_3019 = new
        JScrollPane(stepArea_3019);
74)
75)         // tambahkan panel ke frame
76)         add(inputPanel_3019,
        BorderLayout.NORTH);
77)         add(panelArray_3019,
        BorderLayout.CENTER);
78)         add(controlPanel_3019,
        BorderLayout.SOUTH);
79)         add(scrollPane_3019,
        BorderLayout.EAST);
80)
81)         // event set array
82)         setButton_3019.addActionListener(e ->
        setArrayFromInput_3019());
83)
84)         // event selanjutnya
85)         stepButton_3019.addActionListener(e ->
        performStep_3019());
86)
87)         // event reset
88)         resetButton_3019.addActionListener(e ->
        reset_3019());
89)     }
90)
91)     private void setArrayFromInput_3019() {
92)         String text_3019 =
        inputField_3019.getText().trim();
93)         if (text_3019.isEmpty()) return;
94)         String[] parts_3019 =
        text_3019.split(",");
95)         array_3019 = new
        int[parts_3019.length];
96)         try {
97)             for (int k = 0; k <
        parts_3019.length; k++) {

```

```

98)         array_3019[k] =
Integer.parseInt(parts_3019[k].trim());
99)     }
100)    } catch (NumberFormatException
e) {
101)        JOptionPane.showMessageDialog(this, "Masukkan
        hanya angka dengan koma!",
102)        "Error",
        JOptionPane.ERROR_MESSAGE);
103)        return;
104)    }
105)
106)        labelArray_3019 = new
JLabel[array_3019.length];
107)        panelArray_3019.removeAll();
108)        for (int k = 0; k <
array_3019.length; k++) {
109)            labelArray_3019[k] = new
JLabel(String.valueOf(array_3019[k]));
110)            labelArray_3019[k].setFont(new Font("Arial",
Font.BOLD, 24));
111)            labelArray_3019[k].setOpaque(true);
112)            labelArray_3019[k].setBackground(Color.WHITE)
;
113)            labelArray_3019[k].setBorder(BorderFactory.cr
eateLineBorder(Color.BLACK));
114)            labelArray_3019[k].setPreferredSize(new
Dimension(50, 50));
115)            labelArray_3019[k].setHorizontalAlignment(Swi
ngConstants.CENTER);
116)            panelArray_3019.add(labelArray_3019[k]);
117)        }
118)
119)            mergeQueue_3019.clear();
120)            generateMergeSteps_3019(0,
array_3019.length - 1);
121)            stepButton_3019.setEnabled(true);
122)            stepArea_3019.setText("");
123)            stepCount_3019 = 1;
124)            isMerging_3019 = false;
125)
126)            panelArray_3019.revalidate();
127)            panelArray_3019.repaint();
128)        }
129)
130)        private void
generateMergeSteps_3019(int left, int right) {
131)            if (left >= right) return;

```

```

132)             int mid = (left + right) / 2;
133)             generateMergeSteps_3019(left,
            mid);
134)             generateMergeSteps_3019(mid + 1,
            right);
135)             mergeQueue_3019.add(new
            int[] {left, mid, right});
136)         }
137)
138)         private void performStep_3019() {
139)             resetHighlights_3019();
140)
141)             if (!isMerging_3019 &&
            !mergeQueue_3019.isEmpty()) {
142)                 int[] range_3019 =
            mergeQueue_3019.poll();
143)                 left_3019 = range_3019[0];
144)                 mid_3019 = range_3019[1];
145)                 right_3019 =
            range_3019[2];
146)                 temp_3019 = new
            int[right_3019 - left_3019 + 1];
147)                 i_3019 = left_3019;
148)                 j_3019 = mid_3019 + 1;
149)                 k_3019 = 0;
150)                 copying_3019 = false;
151)                 isMerging_3019 = true;
152)
            stepArea_3019.append("Langkah " +
            stepCount_3019++ +
153)                                ": Mulai
            merge dari " + left_3019 + " ke " + right_3019 +
            "\n");
154)             return;
155)         }
156)
157)         if (isMerging_3019 &&
            !copying_3019) {
158)             if (i_3019 <= mid_3019 &&
            j_3019 <= right_3019) {
159)                 labelArray_3019[i_3019].setBackground(Color.C
            YAN);
160)                 labelArray_3019[j_3019].setBackground(Color.C
            YAN);
161)                 if
            (array_3019[i_3019] <= array_3019[j_3019]) {
162)                     temp_3019[k_3019++] = array_3019[i_3019++];
163)                 } else {
164)                     temp_3019[k_3019++] = array_3019[j_3019++];
165)                 }
166)                 stepArea_3019.append("Langkah " +

```

```

        stepCount_3019++ + ": Bandingkan dan salin
        elemen\n");
167)                                return;
168)                                } else if (i_3019 <=
        mid_3019) {
169)                                temp_3019[k_3019++]
        = array_3019[i_3019++];
170)                                stepArea_3019.append("Langkah " +
        stepCount_3019++ + ": Salin sisa kiri\n");
171)                                return;
172)                                } else if (j_3019 <=
        right_3019) {
173)                                temp_3019[k_3019++]
        = array_3019[j_3019++];
174)                                stepArea_3019.append("Langkah " +
        stepCount_3019++ + ": Salin sisa kanan\n");
175)                                return;
176)                                } else {
177)                                copying_3019 =
        true;
178)                                k_3019 = 0;
179)                                return;
180)                                }
181)                                }
182)
183)                                if (copying_3019 && k_3019 <
        temp_3019.length) {
184)                                array_3019[left_3019 +
        k_3019] = temp_3019[k_3019];
185)                                labelArray_3019[left_3019
        +
        k_3019].setText(String.valueOf(temp_3019[k_3019]));
186)                                labelArray_3019[left_3019
        + k_3019].setBackground(Color.GREEN);
187)                                k_3019++;
188)                                stepArea_3019.append("Langkah " +
        stepCount_3019++ + ": Tempelkan ke array utama\n");
189)                                return;
190)                                }
191)
192)                                if (copying_3019 && k_3019 ==
        temp_3019.length) {
193)                                isMerging_3019 = false;
194)                                copying_3019 = false;
195)                                }
196)
197)                                if (mergeQueue_3019.isEmpty() &&
        !isMerging_3019) {
198)                                stepArea_3019.append("Selesai.\n");
199)                                stepButton_3019.setEnabled(false);

```



```

200)        JOptionPane.showMessageDialog(this, "Merge
           Sort selesai!");
201)    }
202)    }
203)
204)        private void resetHighlights_3019() {
205)            if (labelArray_3019 == null)
           return;
206)            for (JLabel label :
           labelArray_3019) {
207)                label.setBackground(Color.WHITE);
208)            }
209)        }
210)
211)        private void reset_3019() {
212)            inputField_3019.setText("");
213)            panelArray_3019.removeAll();
214)            panelArray_3019.revalidate();
215)            panelArray_3019.repaint();
216)            stepArea_3019.setText("");
217)
           stepButton_3019.setEnabled(false);
218)            mergeQueue_3019.clear();
219)            isMerging_3019 = false;
220)            stepCount_3019 = 1;
221)        }
222)
223)        public static void main(String[] args)
           {
224)            SwingUtilities.invokeLater(() ->
           {
225)                MergeSortGUI_2511533019
           gui = new MergeSortGUI_2511533019();
226)                gui.setVisible(true);
227)            });
228)        }
229)    }

```

Program kelima yaitu MergeSortGUI_2511533019 merupakan implementasi algoritma Merge Sort dalam bentuk GUI. Struktur antarmuka yang digunakan juga mengadopsi program InsertSortGUI pada praktikum sebelumnya, sehingga komponen GUI yang digunakan relatif sama. Perbedaan utama terletak pada logika pengurutan yang menerapkan metode Divide and Conquer.

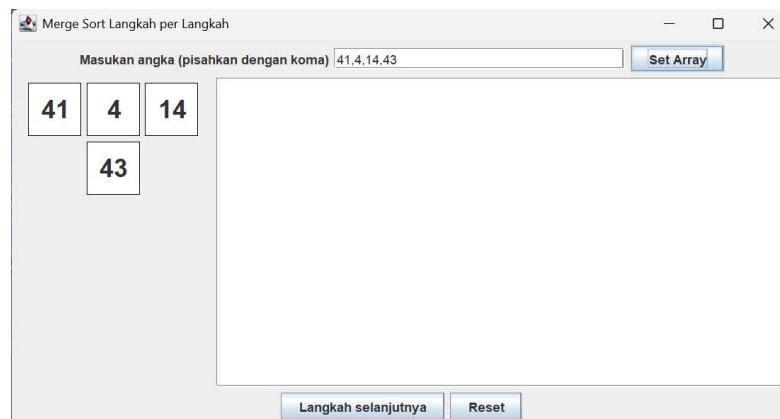
Method setArrayFromInput_3019() digunakan untuk membaca input dari user, mengubahnya menjadi array integer, dan menampilkan data ke dalam bentuk visual. Setelah itu, method

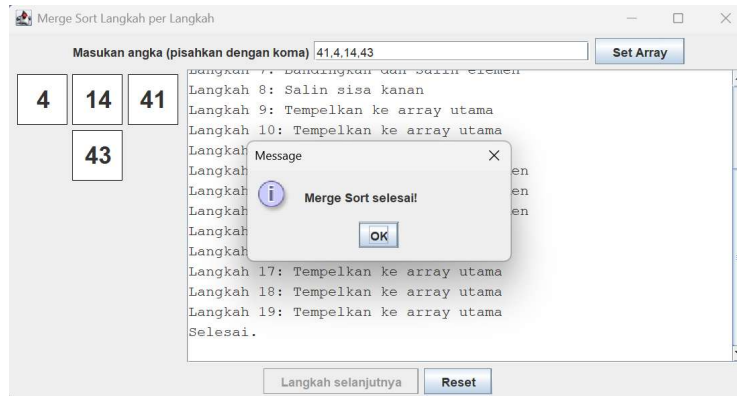
generateMergeSteps_3019() digunakan untuk menentukan urutan proses pembagian array yang akan dilakukan selama algoritma Merge Sort berjalan. Proses pengurutan selanjutnya dilakukan secara bertahap melalui method performStep_3019(), yang bertugas menampilkan proses perbandingan, penggabungan, dan penyalinan data ke array utama. Setiap langkah divisualisasikan menggunakan perubahan warna pada elemen serta dicatat pada area log. Dengan demikian, user dapat mengamati secara langsung proses kerja algoritma Merge Sort dari tahap pembagian hingga seluruh data berhasil terurut.

- 3) Run Program. Setelah di run, Pada awal keluar tampilan untuk bisa melakukan pengurutan angka secara instan dengan metode Merge Sort.

Nah sebagai user inputkan beberapa nilai/angka acak yang ingin kita cari urutan nilai/angkanya. Lalu klik tombol untuk set array dan klik tombol langkah selanjutnya hingga keluar tampilan bahwa sorting selesai.

Sebagai contoh saya memasukkan nilai sebagai berikut:





2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A Z) atau Durasi (terpendek ke terpanjang).

2.2.2 Kelas ADT Lagu

- 1) Klik kanan package Pekan8 dan pilih “New”, pilih class. Lalu buat nama “Lagu_2511533019”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```

1) package Pekan8_2511533019;
2)
3) public class Lagu_2511533019 {
4)     String judul_3019;
5)     String penyanyi_3019;
6)     int durasi_3019;;
7)
8)     public Lagu_2511533019(String judul_3019, String
        penyanyi_3019, int durasi_3019) {
9)         this.judul_3019=judul_3019;;
10)        this.penyanyi_3019=penyanyi_3019;;
11)        this.durasi_3019=durasi_3019;
12)    }
13) }

```

Program ini dibuat untuk digunakan sebagai kelas Lagu untuk menyimpan data lagu. Pada class ini terdapat 2 atribut bertipe String, yaitu judul_3019, penyanyi_3019, dan 1 atribut bertipe

integer yaitu `durasi_3019` untuk menyimpan judul, penyanyi serta durasi dari lagu.

Class ini memiliki constructor yang digunakan untuk menginisialisasi nilai atribut `judul_3019`, `penyanyi_3019`, dan `durasi_3019` saat objek dibuat. Dengan adanya class ini, data lagu dapat disimpan dalam bentuk objek sehingga memudahkan proses pengelolaan dan pengurutan data menggunakan algoritma Shell Sort, Quick Sort, dan Merge Sort pada class `Sorting_2511533019`.

Class `Lagu_2511533019` berperan sebagai struktur data utama yang digunakan untuk menyimpan informasi setiap lagu yang nantinya akan diproses dan diurutkan berdasarkan judul maupun durasi sesuai algoritma yang dipilih user.

2.2.3 Kelas Utama

- 1) Klik kanan package `Pekan8` dan pilih “New”, pilih class. Lalu buat nama “`Sorting_2511533019`”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package Pekan8_2511533019;
2)
3) import java.util.Scanner;
4)
5) public class Sorting_2511533019 {
6)
7)     // array menyimpan data lagu
8)     static Lagu_2511533019[] dataLagu_3019 = new
       Lagu_2511533019[20];
9)     // byk lagu yang digunakan
10)    static int jumlahData_3019 = 7;
11)    // Method mengisi data lagu
12)    public static void inputData_3019() {
13)        dataLagu_3019[0] = new
       Lagu_2511533019("Joyride", "Cortis", 195);
14)        dataLagu_3019[1] = new
       Lagu_2511533019("Almost Is Never Enough", "Ariana
       Grande", 327);
15)        dataLagu_3019[2] = new
       Lagu_2511533019("Merry Christmas, I Miss You",
       "Alex Crichton", 247);
16)        dataLagu_3019[3] = new
       Lagu_2511533019("Wonderwall", "Oasis", 258);
```

```

17)     dataLagu_3019[4] = new Lagu_2511533019("Acai
      Bowl", "Cortis", 210);
18)     dataLagu_3019[5] = new
      Lagu_2511533019("Monokrom", "Tulus", 230);
19)     dataLagu_3019[6] = new
      Lagu_2511533019("Komang", "Raim Laode", 220);
20)     }
21)     // method untuk menampilkan data lagu
22)     public static void tampilData_3019() {
23)         for (int i_3019 = 0; i_3019 <
      jumlahData_3019; i_3019++) {
24)             System.out.println((i_3019 + 1) + ". "+
      dataLagu_3019[i_3019].judul_3019 + " - "
25)                 +
      dataLagu_3019[i_3019].penyanyi_3019 + " - "
26)                 +
      dataLagu_3019[i_3019].durasi_3019 + " detik");
27)         }
28)     }
29)     // shell sort
30)     public static void shellSort_3019() {
31)         int n_3019 = jumlahData_3019;
32)         // nentuin gap
33)         for (int gap_3019 = n_3019 / 2; gap_3019 >
      0; gap_3019 /= 2) {
34)             for (int i_3019 = gap_3019; i_3019 <
      n_3019; i_3019++) {
35)                 Lagu_2511533019 temp_3019 =
      dataLagu_3019[i_3019];
36)                 int j_3019 = i_3019;
37)
38)                 // membandingkan judul lagu
39)                 while (j_3019 >= gap_3019 &&
40)                     dataLagu_3019[j_3019 -
      gap_3019].judul_3019.compareToIgnoreCase(temp_3019.
      judul_3019) > 0) {
41)                     dataLagu_3019[j_3019] =
      dataLagu_3019[j_3019 - gap_3019];
42)                     j_3019 -= gap_3019;
43)                 }
44)                 dataLagu_3019[j_3019] = temp_3019;
45)             }
46)         }
47)     }
48)
49)     // quick sort
50)     public static int partition_3019(int low_3019,
      int high_3019) {
51)         int pivot_3019 =
      dataLagu_3019[high_3019].durasi_3019;
52)         int i_3019 = low_3019 - 1;

```

```

53)         for (int j_3019 = low_3019; j_3019 <
             high_3019; j_3019++) {
54)             // Membandingkan durasi lagu dengan
             pivot
55)             if (dataLagu_3019[j_3019].durasi_3019 <
                 pivot_3019) {
56)                 i_3019++;
57)                 Lagu_2511533019 temp_3019 =
                 dataLagu_3019[i_3019];
58)                 dataLagu_3019[i_3019] =
                 dataLagu_3019[j_3019];
59)                 dataLagu_3019[j_3019] = temp_3019;
60)             }
61)         }
62)
63)         // Menempatkan pivot pada posisi yang benar
64)         Lagu_2511533019 temp_3019 =
         dataLagu_3019[i_3019 + 1];
65)         dataLagu_3019[i_3019 + 1] =
         dataLagu_3019[high_3019];
66)         dataLagu_3019[high_3019] = temp_3019;
67)         return i_3019 + 1;
68)     }
69)     // Method quick sort
70)     public static void quickSort_3019(int low_3019,
        int high_3019) {
71)         if (low_3019 < high_3019) {
72)             int pi_3019 = partition_3019(low_3019,
                high_3019);
73)             quickSort_3019(low_3019, pi_3019 - 1);
74)             quickSort_3019(pi_3019 + 1, high_3019);
75)         }
76)     }
77)     //merge sort
78)     public static void merge_3019(int kiri_3019,
        int tengah_3019, int kanan_3019) {
79)         int n1_3019 = tengah_3019 - kiri_3019 + 1;
80)         int n2_3019 = kanan_3019 - tengah_3019;
81)         Lagu_2511533019[] L_3019 = new
            Lagu_2511533019[n1_3019];
82)         Lagu_2511533019[] R_3019 = new
            Lagu_2511533019[n2_3019];
83)
84)         for (int i_3019 = 0; i_3019 < n1_3019;
             i_3019++) {
85)             L_3019[i_3019] =
             dataLagu_3019[kiri_3019 + i_3019];
86)         }
87)         for (int j_3019 = 0; j_3019 < n2_3019;
             j_3019++) {

```

```

88)         R_3019[j_3019] =
            dataLagu_3019[tengah_3019 + 1 + j_3019];
89)     }
90)     int i_3019 = 0;
91)     int j_3019 = 0;
92)     int k_3019 = kiri_3019;
93)
94)     while (i_3019 < n1_3019 && j_3019 <
            n2_3019) {
95)
96)         if
            (L_3019[i_3019].judul_3019.compareToIgnoreCase(
97)             R_3019[j_3019].judul_3019) <=
            0) {
98)             dataLagu_3019[k_3019] =
                L_3019[i_3019];
99)             i_3019++;
100)        } else {
101)            dataLagu_3019[k_3019] =
                R_3019[j_3019];
102)            j_3019++;
103)        }
104)        k_3019++;
105)    }
106)    // Menyalin sisa elemen array kiri
107)    while (i_3019 < n1_3019) {
108)        dataLagu_3019[k_3019++] =
            L_3019[i_3019++];
109)    }
110)    // Menyalin sisa elemen array kanan
111)    while (j_3019 < n2_3019) {
112)        dataLagu_3019[k_3019++] =
            R_3019[j_3019++];
113)    }
114)    }
115)    // Method Merge Sort
116)    public static void mergeSort_3019(int
        kiri_3019, int kanan_3019) {
117)        if (kiri_3019 < kanan_3019) {
118)            int tengah_3019 = (kiri_3019 +
                kanan_3019) / 2;
119)            mergeSort_3019(kiri_3019,
                tengah_3019);
120)            mergeSort_3019(tengah_3019 + 1,
                kanan_3019);
121)            merge_3019(kiri_3019,
                tengah_3019, kanan_3019);
122)        }
123)    }
124)    //main program

```

```

125)         public static void main(String[] args) {
126)             Scanner input_3019 = new
                Scanner(System.in);
127)             inputData_3019();
128)             // Menampilkan menu pilihan algoritma
129)             System.out.println("=== Sorting
                Playlist Lagu ===");
130)             System.out.println("1. Shell Sort
                (Judul A-Z)");
131)             System.out.println("2. Quick Sort
                (Durasi Asc)");
132)             System.out.println("3. Merge Sort
                (Judul A-Z)");
133)             System.out.print("Pilih Algoritma :
                ");
134)             int pilihan_3019 =
                input_3019.nextInt();
135)             // Menampilkan data sebelum sorting
136)             System.out.println("\nData Sebelum
                Sorting:");
137)             tampilData_3019();
138)             switch (pilihan_3019) {
139)                 case 1:
140)                     shellSort_3019();
141)                     System.out.println("\nData
                        Setelah Shell Sort:");
142)                     break;
143)                 case 2:
144)                     quickSort_3019(0,
                        jumlahData_3019 - 1);
145)                     System.out.println("\nData
                        Setelah Quick Sort:");
146)                     break;
147)                 case 3:
148)                     mergeSort_3019(0,
                        jumlahData_3019 - 1);
149)                     System.out.println("\nData
                        Setelah Merge Sort:");
150)                     break;
151)                 default:
152)                     System.out.println("Pilihan
                        tidak valid!");
153)                     return;
154)             }
155)             // Menampilkan hasil sorting
156)             tampilData_3019();
157)             input_3019.close();
158)         }
159)     }

```


Kelas `Sorting_2511533019` digunakan untuk mengimplementasikan tiga algoritma pengurutan yaitu Shell Sort, Quick Sort, dan Merge Sort pada data playlist lagu. Data lagu disimpan dalam array objek `Lagu_2511533019` yang memiliki atribut judul lagu, penyanyi, dan durasi lagu. Pada program ini sesuai dengan perintah tugas, user dapat memilih algoritma sorting yang ingin digunakan melalui menu yang ditampilkan pada saat program dijalankan.

Method `inputData_3019()` berfungsi untuk mengisi data lagu ke dalam array `dataLagu_3019`. Data yang dimasukkan terdiri dari tujuh lagu yang memiliki judul, penyanyi, dan durasi yang berbeda. Selanjutnya, method `tampilData_3019()` digunakan untuk menampilkan seluruh data lagu yang tersimpan pada array.

Pada algoritma Shell Sort, method `shellSort_3019()` digunakan untuk mengurutkan data lagu berdasarkan judul secara alfabetis (A-Z). Proses pengurutan dilakukan menggunakan nilai gap yang awalnya bernilai setengah dari jumlah data. Nilai gap akan terus diperkecil hingga bernilai satu. Variabel `temp_3019` digunakan sebagai penyimpanan sementara, sedangkan variabel `j_3019` digunakan untuk mencari posisi yang tepat saat proses pergeseran elemen dilakukan.

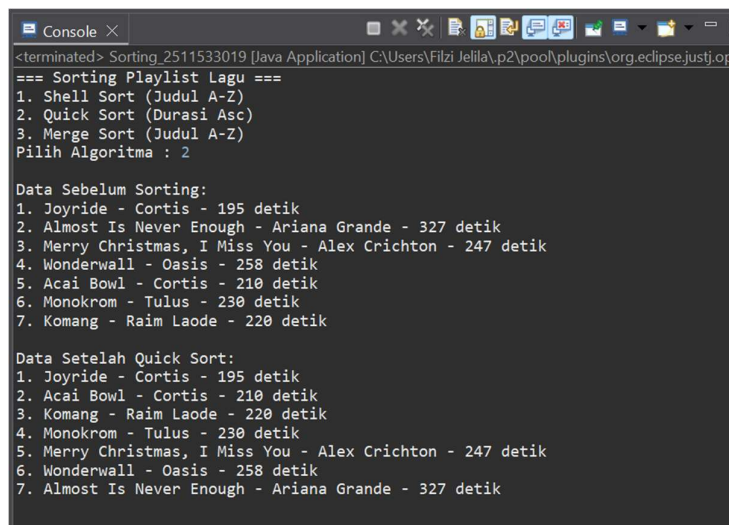
Pada algoritma Quick Sort, method `quickSort_3019()` digunakan untuk mengurutkan lagu berdasarkan durasi secara ascending (terkecil ke terbesar). Method ini memanfaatkan method `partition_3019()` untuk menentukan posisi pivot. Variabel `pivot_3019` digunakan sebagai nilai acuan perbandingan, sedangkan variabel `i_3019` dan `j_3019` digunakan untuk mengatur proses pertukaran data yang memiliki nilai durasi lebih kecil dari pivot. Setelah proses partisi selesai, method `quickSort_3019()` dipanggil secara rekursif hingga seluruh data terurut.

Pada algoritma Merge Sort, method `mergeSort_3019()` digunakan untuk mengurutkan data lagu berdasarkan judul secara alfabetis (A-Z). Algoritma ini bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil menggunakan konsep divide and conquer. Selanjutnya method `merge_3019()` digunakan untuk menggabungkan kembali bagian-bagian tersebut dalam keadaan terurut. Array sementara `L_3019` dan `R_3019` digunakan untuk menyimpan data selama proses penggabungan berlangsung.

Pada method `main()`, program terlebih dahulu memanggil method `inputData_3019()` untuk mengisi data lagu. Setelah itu program menampilkan menu pilihan algoritma sorting yang dapat dipilih user. Data lagu akan ditampilkan sebelum proses pengurutan dilakukan. Berdasarkan pilihan user, program akan menjalankan salah satu algoritma yaitu Shell Sort, Quick Sort, atau Merge Sort, kemudian menampilkan hasil pengurutan yang telah dilakukan.

3) Run program , berikut contoh output dengan 3 pilihan yang saya masukkan sebagai user:

- quick Sort :



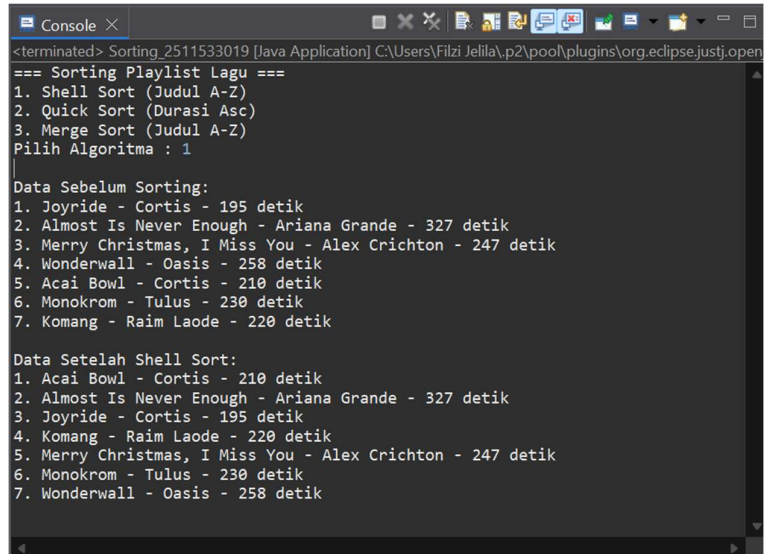
```
<terminated> Sorting_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\org.eclipse.justjop
=== Sorting Playlist Lagu ===
1. Shell Sort (Judul A-Z)
2. Quick Sort (Durasi Asc)
3. Merge Sort (Judul A-Z)
Pilih Algoritma : 2

Data Sebelum Sorting:
1. Joyride - Cortis - 195 detik
2. Almost Is Never Enough - Ariana Grande - 327 detik
3. Merry Christmas, I Miss You - Alex Crichton - 247 detik
4. Wonderwall - Oasis - 258 detik
5. Acai Bowl - Cortis - 210 detik
6. Monokrom - Tulus - 230 detik
7. Komang - Raim Laode - 220 detik

Data Setelah Quick Sort:
1. Joyride - Cortis - 195 detik
2. Acai Bowl - Cortis - 210 detik
3. Komang - Raim Laode - 220 detik
4. Monokrom - Tulus - 230 detik
5. Merry Christmas, I Miss You - Alex Crichton - 247 detik
6. Wonderwall - Oasis - 258 detik
7. Almost Is Never Enough - Ariana Grande - 327 detik
```

Diurutkan dari durasi terkecil-terlama

- shell Sort :



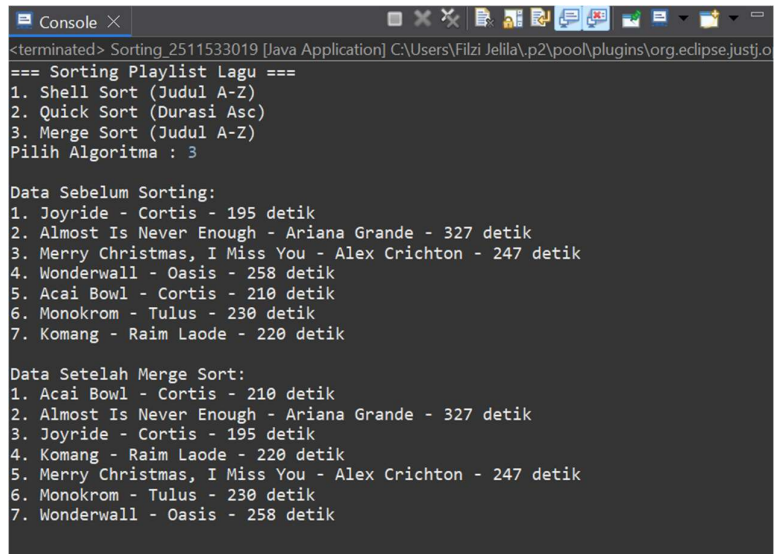
```
<terminated> Sorting_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\org.eclipse.justj.open
=== Sorting Playlist Lagu ===
1. Shell Sort (Judul A-Z)
2. Quick Sort (Durasi Asc)
3. Merge Sort (Judul A-Z)
Pilih Algoritma : 1

Data Sebelum Sorting:
1. Joyride - Cortis - 195 detik
2. Almost Is Never Enough - Ariana Grande - 327 detik
3. Merry Christmas, I Miss You - Alex Crichton - 247 detik
4. Wonderwall - Oasis - 258 detik
5. Acai Bowl - Cortis - 210 detik
6. Monokrom - Tulus - 230 detik
7. Komang - Raim Laode - 220 detik

Data Setelah Shell Sort:
1. Acai Bowl - Cortis - 210 detik
2. Almost Is Never Enough - Ariana Grande - 327 detik
3. Joyride - Cortis - 195 detik
4. Komang - Raim Laode - 220 detik
5. Merry Christmas, I Miss You - Alex Crichton - 247 detik
6. Monokrom - Tulus - 230 detik
7. Wonderwall - Oasis - 258 detik
```

Diurutkan sesuai abjad Judul (A-Z)

- merge Sort :



```
<terminated> Sorting_2511533019 [Java Application] C:\Users\Filzi Jelila\p2\pool\plugins\org.eclipse.justj.o
=== Sorting Playlist Lagu ===
1. Shell Sort (Judul A-Z)
2. Quick Sort (Durasi Asc)
3. Merge Sort (Judul A-Z)
Pilih Algoritma : 3

Data Sebelum Sorting:
1. Joyride - Cortis - 195 detik
2. Almost Is Never Enough - Ariana Grande - 327 detik
3. Merry Christmas, I Miss You - Alex Crichton - 247 detik
4. Wonderwall - Oasis - 258 detik
5. Acai Bowl - Cortis - 210 detik
6. Monokrom - Tulus - 230 detik
7. Komang - Raim Laode - 220 detik

Data Setelah Merge Sort:
1. Acai Bowl - Cortis - 210 detik
2. Almost Is Never Enough - Ariana Grande - 327 detik
3. Joyride - Cortis - 195 detik
4. Komang - Raim Laode - 220 detik
5. Merry Christmas, I Miss You - Alex Crichton - 247 detik
6. Monokrom - Tulus - 230 detik
7. Wonderwall - Oasis - 258 detik
```

Diurutkan sesuai abjad Judul (A-Z)

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Shell Sort, Quick Sort, dan Merge Sort dapat digunakan untuk mengurutkan data secara efektif sesuai kriteria yang ditentukan. Pada praktikum ini, ketiga algoritma berhasil diimplementasikan dalam bahasa pemrograman Java untuk mengurutkan data numerik maupun objek berupa data lagu.

Shell Sort digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis (A-Z) dengan memanfaatkan nilai gap yang semakin kecil hingga seluruh data terurut. Quick Sort digunakan untuk mengurutkan data berdasarkan durasi lagu secara ascending, dengan memanfaatkan teknik partition dan pivot untuk membagi data menjadi beberapa bagian. Sedangkan Merge Sort digunakan untuk mengurutkan data berdasarkan judul lagu secara alfabetis menggunakan metode divide and conquer yang membagi data menjadi bagian-bagian kecil kemudian menggabungkannya kembali dalam keadaan terurut.

Melalui praktikum ini, dapat dipahami cara kerja masing-masing algoritma sorting, perbedaan mekanisme yang digunakan, serta penerapannya pada objek dan data nyata. Selain itu, praktikum ini juga meningkatkan pemahaman mengenai penggunaan array, objek, method, rekursi, serta implementasi algoritma pengurutan dalam pemrograman Java.

DAFTAR PUSTAKA

- [1] Wahyudi, “*Sorting*” PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2025.
- [2] Oracle Corporation, Java Platform, Standard Edition Documentation. Oracle, 2025

sssssss